



Sistemas Operacionais

Giovani Delgado Torres

Bruno Bertolini Maluf

Guilherme Ferrari

Guilherme Serafim

Mateus Belloto

Lista de Exercícios: Gestão de Processos e Integração de Sistemas

**Sorocaba/SP
2026**

1 INTRODUCAO	3
2 OBJETIVO DO EXPERIMENTO	3
3 MATERIAIS UTILIZADOS	3
4 PROCEDIMENTO EXPERIMENTAL	3
5 RESULTADOS	5
6 CONCLUSAO	8
REFERÊNCIAS	8

1 INTRODUÇÃO

Este relatório tem como objetivo apresentar os resultados obtidos na realização de uma lista de exercícios voltada à gestão de processos em sistemas operacionais Linux. Foram utilizados diversos comandos do terminal para visualização, monitoramento, controle de processos e análise do sistema de arquivos, permitindo compreender na prática como o sistema operacional gerencia a execução de programas e recursos.

2 OBJETIVO DO EXPERIMENTO

O objetivo desta atividade foi compreender o funcionamento da gestão de processos em sistemas Linux, incluindo identificação de processos, monitoramento em tempo real, envio de sinais, além da integração com conceitos como concorrência, permissões e virtualização.

3 MATERIAIS UTILIZADOS

Para a realização da atividade foram utilizados:

- Sistema operacional Linux (Ubuntu via WSL)
- Terminal Linux
- Comandos: ps, top, kill, pstree, pidof, find, free
- Editor de texto
- Ambiente virtualizado

4 PROCEDIMENTO EXPERIMENTAL

1.

Ao executar o comando, são exibidas colunas como PID, TTY, TIME e CMD. O PID representa o identificador único de cada processo no sistema.

```
$ ps
```

2.

A coluna START mostra quando o processo foi iniciado. Isso ajuda o administrador a identificar há quanto tempo um processo está rodando.

```
$ ps -u guilherme
```

3

3.

O comando `ps -e` mostra todos os processos do sistema, enquanto o `ps` comum mostra apenas os do terminal atual.

```
$ ps -e
```

4.

Com esse comando, é possível descobrir o PID de um processo específico pelo nome.

```
$ pidof bash
```

5.

O `pstree` mostra os processos em forma de árvore, onde processos filhos ficam ligados aos seus processos pais.

```
$ pstree
```

6.

Aqui conseguimos ver os PIDs junto com a árvore de processos.

O `systemd` (ou `init`) é o PID 1 porque é o primeiro processo iniciado pelo sistema.

```
$ pstree -p
```

7.

Os estados dos processos indicam o que eles estão fazendo.

"S" significa em espera (sleeping) e "R" significa em execução (running).

```
$ ps -el
```

8.

O estado "D" indica que o processo está aguardando acesso ao hardware, geralmente operações de disco (entrada/saída).

Parte 2: Monitoramento em Tempo Real

9.

O comando top mostra os processos em tempo real, atualizando automaticamente. \$ top

10.

Ao pressionar "M", os processos passam a ser ordenados pelo uso de memória.

```
$ top -d 5
```

11.

Esse comando altera o tempo de atualização da tela.

12.

Permite monitorar apenas um processo específico.

```
$ top -p PID
```

13.

Ao pressionar "k", o sistema pede o PID e o sinal que será enviado ao processo.

14.

Para sair do top, basta pressionar a tecla "q".

Parte 3: Controle e Sinais

15.

O sinal STOP pausa o processo, enquanto o KILL encerra imediatamente.

16.

Força o encerramento do processo.

```
$ kill -9 PID
```

17.

Permite encerrar processos pelo nome.

```
$ killall firefox
```

18.

O cursor vira um "X" e permite clicar em uma janela para fechá-la.

```
$ xkill
```

19.

Ao tentar matar um processo do root sem sudo, aparece erro de permissão, pois usuários comuns não têm acesso administrativo.

Parte 4: Sistema /proc

20.

As pastas com números representam os PIDs dos processos ativos.

```
$ cd /proc
```

21.

Cada pasta contém informações do processo correspondente.

```
$ cd /proc/PID $ ls
```

22.

Os arquivos ocupam 0 bytes porque são gerados em tempo real pelo sistema.

Parte 5: Integração

23.

O sistema divide o tempo da CPU entre processos usando escalonamento, alternando rapidamente entre eles.

24.

Qualquer usuário pode executar o comando, pois possui permissão de execução.

```
$ which ps $ ls -l /usr/bin/ps
```

25.

Mostra apenas a memória reservada para a máquina virtual.

```
$ free -m
```

26.

Lista os processos que mais consomem memória.

```
$ ps aux --sort=-%mem | head > uso_memoria.txt
```

27.

Localiza onde está o manual do comando.

```
$ find / -name kill
```

28.

Cria um loop infinito e permite pausar o processo.

```
$ while true; do echo "teste"; done $ kill -STOP PID
```

29.

Após mudar para root, o usuário comum não consegue controlar o processo.

```
$ chown root script.sh
```

30.

Mostra informações do processador.

```
$ cat /proc/cpuinfo
```

5 RESULTADOS

Durante a execução dos exercícios, foi possível compreender na prática como o sistema operacional gerencia processos, desde sua criação até finalização. Também foi possível observar como o sistema organiza processos em árvore, como realiza o compartilhamento de CPU e como controla permissões de execução.

O uso de ferramentas como top e ps permitiu visualizar o comportamento do sistema em tempo real, enquanto comandos como kill demonstraram o controle direto sobre processos.

6 CONCLUSÃO

A realização desta atividade foi fundamental para consolidar o entendimento sobre gestão de processos em sistemas Linux. Foi possível compreender como o sistema operacional controla a execução de programas, gerencia recursos e aplica

permissões de segurança. Além disso, a prática mostrou a importância de comandos essenciais para monitoramento e administração do sistema, evidenciando o papel do kernel na organização e controle de todas essas operações.

REFERÊNCIAS

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. Sistemas Operacionais.

TANENBAUM, Andrew S. Sistemas Operacionais Modernos.

Documentação oficial do Linux – man pages (man ps, man top, man kill).